
PROJECT IMPLEMENTATION REPORT

Digital Banking CI/CD Platform on AWS

Jenkins | GitHub | Maven | Docker | Amazon ECR | Ansible | Amazon EKS |
Kubernetes | Prometheus | Grafana

Author: Jeni Balar

Role: Cloud / DevOps Infrastructure Project

Repository: [View Project On GitHub](#)

Core Cloud: AWS (VPC, EC2, IAM, NAT Gateway, Security Groups, Internet Gateway, LoadBalancer)

Application: Java | Spring Boot | Maven

Infrastructure: Jenkins | GitHub | Maven | Docker | Amazon ECR | Ansible | Amazon EKS | Kubernetes | Prometheus | Grafana

[EXECUTIVE ABSTRACT]

This technical document outlines the implementation of an AWS-based Digital Banking CI/CD platform using Jenkins, Docker, Amazon ECR, Amazon EKS, Kubernetes, Ansible, Prometheus and Grafana. The project demonstrates an end-to-end DevOps workflow covering automated build, testing, containerization, deployment across Dev, Staging and Production environments, and application monitoring.

Key achievements include:

- Automated CI/CD workflow from GitHub source code to Kubernetes deployment.
- Maven build and TestNG test execution integrated into Jenkins.
- Docker image creation and private Amazon ECR storage.
- Ansible-based deployment automation to Amazon EKS.
- Kubernetes deployment with a single LoadBalancer service for application access.
- Two-AZ VPC design with two public and two private subnets.
- Prometheus and Grafana monitoring for application and Kubernetes metrics.
- End-to-end verification of the application, transaction flow and pipeline stages.

INDEX

SR.No.	TOPIC	PG No.
1	Project Overview	3
2	Objectives	4
3	Architecture	5-7
4	Implementation & Screenshots	8-27
5	Kubernetes and Deployment Design	28
6	Monitoring and Observability	29
7	Key Verification Commands	30
8	Troubleshooting	31
9	End-to-End Verification	31
10	Conclusion	32

1. Project Overview

The Digital Banking project demonstrates an end-to-end, production-style DevOps and CI/CD workflow for a Spring Boot banking application deployed on AWS. The application source code is maintained in GitHub, from where Jenkins automates the delivery pipeline. The pipeline performs source checkout, Maven-based build and TestNG validation, followed by Docker image creation and pushing the application image to a private Amazon ECR repository.

For deployment, Ansible automates the interaction with Amazon EKS and Kubernetes by updating the EKS kubeconfig, validating the deployment environment, ensuring the required Kubernetes namespace exists, applying the deployment manifest, assigning the required immutable ECR image tag, and monitoring the Kubernetes rollout status. The application is deployed progressively across Dev, Staging, and Production environments, with smoke-test validation between environments and a manual approval gate before Production deployment.

The AWS infrastructure is built within a dedicated VPC using CIDR 10.0.0.0/16, spanning two Availability Zones with two public and two private subnets. The EKS worker infrastructure and application workloads run within the private network, while the public network provides the required external connectivity. The application is exposed through a single Kubernetes Service of type LoadBalancer, which provides the external AWS load-balancing endpoint through which users can access the Digital Banking application.

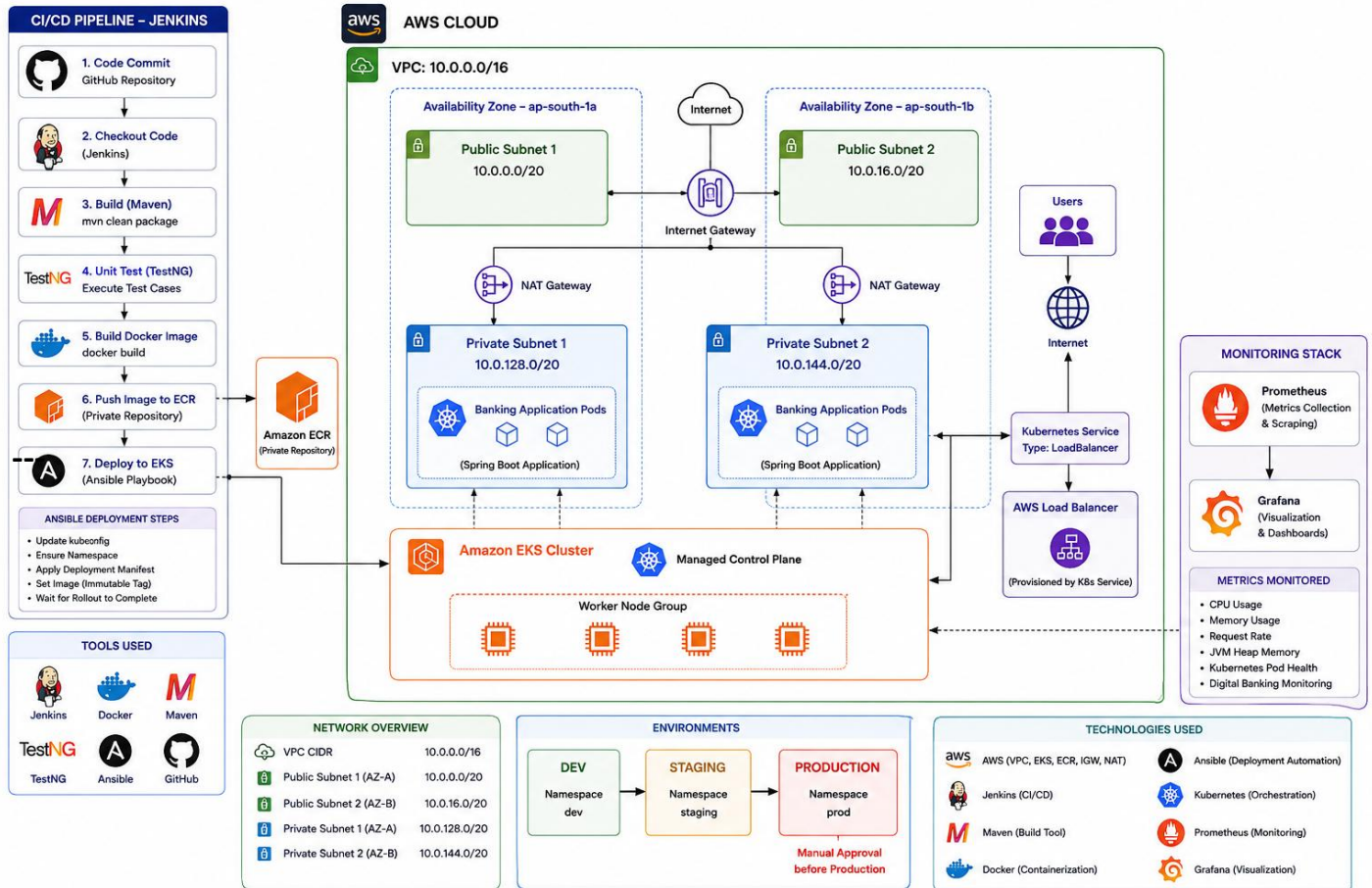
The project also incorporates observability using Prometheus and Grafana, enabling monitoring and visualization of application and Kubernetes metrics, including CPU utilization, memory utilization, request rate, JVM heap memory, and Pod health. Overall, the project demonstrates a complete cloud-based DevOps lifecycle covering source control, continuous integration, automated testing, containerization, image management, Kubernetes deployment, environment promotion, Production release control, application validation, and monitoring.

2. Objectives

The primary objectives of the **Digital Banking CI/CD project** are:

1. **Automate the CI/CD lifecycle** from source-code commit to application deployment using Jenkins.
2. **Implement automated build and testing** using Maven and TestNG to validate application changes before deployment.
3. **Containerize the Spring Boot application** using Docker to provide a consistent and portable deployment package.
4. **Implement secure container image management** using a private Amazon ECR repository.
5. **Deploy and manage the application on Amazon EKS** using Kubernetes for container orchestration.
6. **Automate Kubernetes deployments using Ansible**, reducing manual deployment activities and improving consistency.
7. **Implement separate Dev, Staging, and Production environments** to progressively validate application releases.
8. **Introduce deployment validation and release control** through smoke testing and a manual Production approval gate.
9. **Design a highly available AWS network architecture** using a VPC, two Availability Zones, and public/private subnets.
10. **Expose the application through a Kubernetes LoadBalancer Service** to provide an accessible application endpoint.
11. **Implement application and infrastructure monitoring** using Prometheus and Grafana for operational visibility.
12. **Demonstrate an end-to-end DevOps workflow** covering source control, integration, testing, containerization, deployment, validation, release management, and monitoring.

3. Architecture



3.1 Architecture Explanation

- The Digital Banking project uses a **cloud-native AWS architecture** in which the Spring Boot application is containerized with Docker and deployed on **Amazon EKS using Kubernetes**.
- The infrastructure is deployed inside a dedicated **VPC with CIDR 10.0.0.0/16**, spanning two Availability Zones:
 - AZ-A:** ap-south-1a
 - AZ-B:** ap-south-1b
- The VPC contains four subnets:

Component	Actual Project CIDR / Placement
VPC	10.0.0.0/16
Public Subnet 1	10.0.0.0/20 — ap-south-1a
Public Subnet 2	10.0.16.0/20 — ap-south-1b
Private Subnet 1	10.0.128.0/20 — ap-south-1a
Private Subnet 2	10.0.144.0/20 — ap-south-1b

- The **EKS worker infrastructure and Digital Banking application Pods run in the private subnets**, while the public subnets provide the required internet-facing networking components.
- A **single Kubernetes LoadBalancer Service** provides external access to the application. The application runs as multiple replicas across the EKS worker infrastructure, providing redundancy.

3.2 End-to-End Flow

- User accesses the Digital Banking application through the external AWS Load Balancer endpoint.
- The AWS Load Balancer forwards the request to the **Kubernetes LoadBalancer Service**.
- The Kubernetes Service routes the request to a healthy **Digital Banking Pod** running on Amazon EKS.
- The Spring Boot Digital Banking application processes the request and returns the required banking response.
- The response travels back through the Kubernetes Service and Load Balancer to the user.
- Application and Kubernetes metrics are collected by **Prometheus** and visualized through **Grafana** dashboards.

3.3 Security Flow

- The application workloads run inside the **private subnets** 10.0.128.0/20 and 10.0.144.0/20.
- The external application endpoint is provided through a **single Kubernetes Service of type LoadBalancer**.
- The Kubernetes LoadBalancer provides the controlled entry point for external HTTP traffic to the application.
- EKS worker nodes and application Pods are not directly exposed to the public internet.
- Private-subnet resources use the **NAT Gateway** for required outbound internet connectivity.
- The **Internet Gateway** provides connectivity between the VPC's public networking components and the Internet.

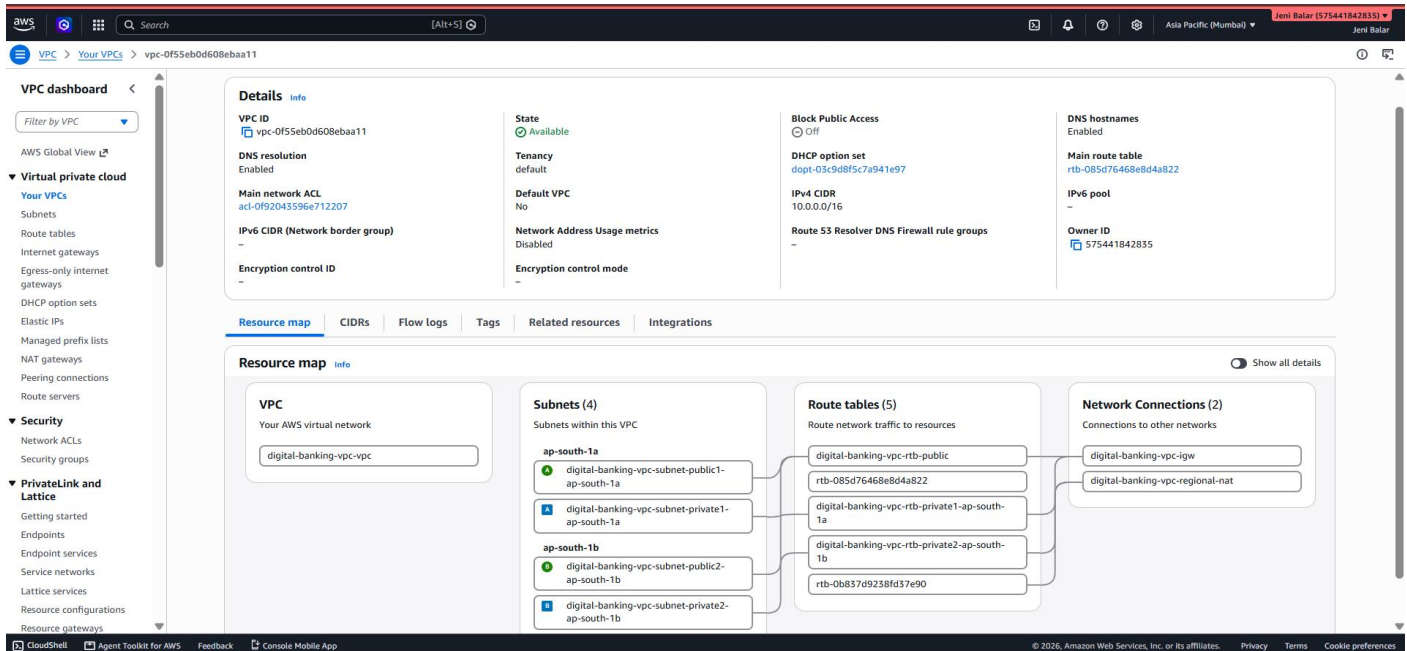
3.4 AWS Services Used

Service / Technology	Purpose
Amazon VPC	Provides the isolated network environment
Public & Private Subnets	Separates internet-facing and application workloads
Internet Gateway	Provides Internet connectivity for the public networking
NAT Gateway	Provides outbound Internet access for private resources
Amazon ECR	Stores private Docker images
Amazon EKS	Runs and manages the Kubernetes environment
EKS Worker Node Group	Provides compute resources for Kubernetes Pods
AWS Load Balancer	Provides external access to the application through the Kubernetes LoadBalancer Service
IAM	Controls access to AWS resources
Security Groups	Controls network traffic for AWS resources
Jenkins	Automates the application delivery pipeline
Java	Programming language of the application
Spring Boot	Framework used to build the Digital Banking application
Maven	Builds, tests, and packages the Java application
TestNG	Executes automated application tests
Docker	Packages the application into a container image
Ansible	Automates EKS/Kubernetes deployment tasks
Kubernetes	Manages application Pods, Deployments, and Services
Prometheus	Collects application and Kubernetes metrics
Grafana	Displays monitoring metrics through dashboards.

4. Implementation & Screenshots

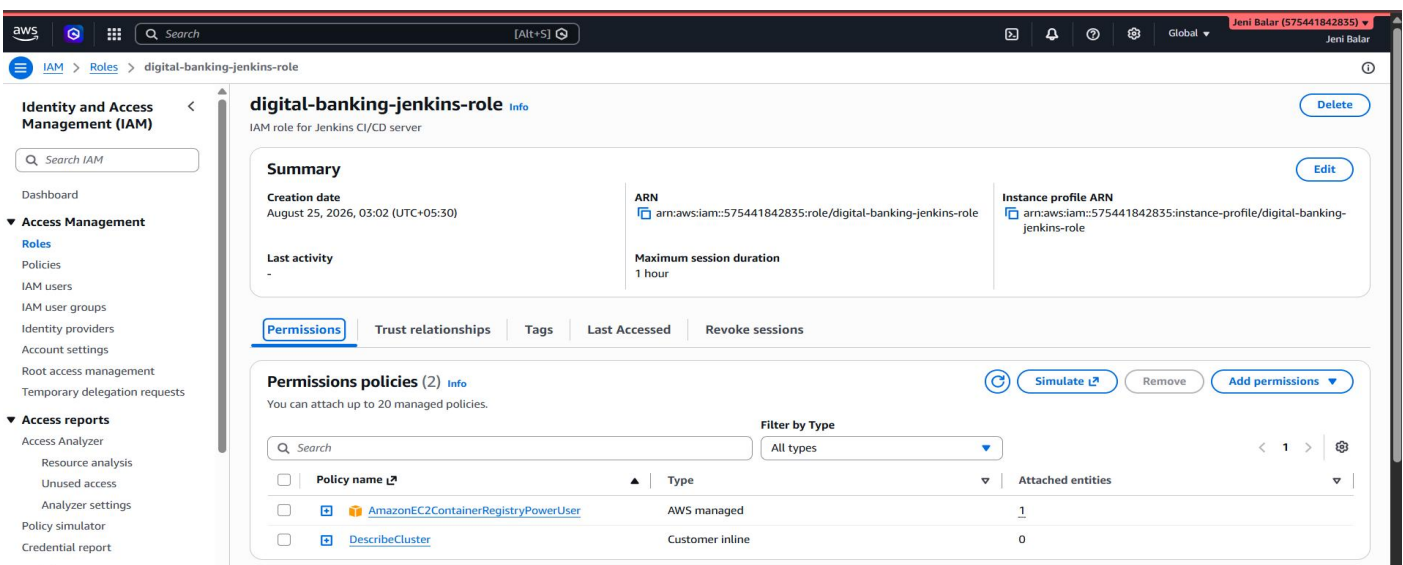
AWS Networking & Jenkins Infrastructure

SS 1 : AWS VPC Network Architecture



- First, I created and verified the Digital Banking VPC with the CIDR block 10.0.0.0/16. The VPC contains four subnets across two Availability Zones, along with the required route tables, Internet Gateway, and NAT Gateway. This provides the network foundation for deploying the Digital Banking application on AWS.

SS 2: Jenkins IAM Role Permissions



- Then I created the digital-banking-jenkins-role IAM role and attached the required permissions for the Jenkins CI/CD server. The role provides access to **Amazon ECR** for managing container images and permission to describe the **Amazon EKS cluster**, allowing Jenkins to interact with the AWS resources required during the CI/CD and deployment process.

SS 3: Jenkins IAM Role Trust Relationship

The screenshot shows the AWS IAM console for the role 'digital-banking-jenkins-role'. The 'Trust relationships' tab is selected, displaying the trust policy JSON:

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Principal": {
7         "Service": "ec2.amazonaws.com"
8       },
9       "Action": "sts:AssumeRole"
10    }
11  ]
12 }
```

The summary section shows the role was created on August 25, 2026, at 03:02 UTC+05:30. The ARN is 'arn:aws:iam::575441842835:role/digital-banking-jenkins-role' and the instance profile ARN is 'arn:aws:iam::575441842835:instance-profile/digital-banking-jenkins-role'. The maximum session duration is 1 hour.

- Then I configured the **trust relationship** for the digital-banking-jenkins-role to allow the **EC2 service** to assume the IAM role using **sts:AssumeRole**. This allows the Jenkins server running on an EC2 instance to use the permissions attached to the role without requiring AWS access keys to be stored on the server.

SS 4: Jenkins EC2 Instance & Configuration

The screenshot shows the AWS EC2 console for the instance 'digital-banking-jenkins' (ID: i-00f81606ad5ec33e2). The instance is in a 'Running' state. The 'Details' tab is selected, showing the instance summary:

- Instance ID: i-00f81606ad5ec33e2
- Public IPv4 address: 65.2.178.72
- Private IPv4 addresses: 10.0.15.121
- Instance state: Running
- Public DNS: ec2-65-2-178-72.ap-south-1.compute.amazonaws.com
- Private IP DNS name (IPv4 only): i-00f81606ad5ec33e2.ap-south-1.compute.internal

The instance is running on an m7i-flex.large instance type in the ap-south-1a availability zone.

- I launched the digital-banking-jenkins **EC2 instance** using the previously created **Digital Banking VPC and subnet**. I associated the digital-banking-sg security group, which was configured with the required **SSH (TCP 22)** and **Jenkins (TCP 8080)** access rules, and attached the digital-banking-jenkins-role IAM role. This EC2 instance was used to host the Jenkins CI/CD server.

Jenkins CI/CD Configuration & Pipeline

SS 5: Jenkins Environment & DevOps Tools Setup

```
=====
Jenkins setup completed
=====
Java:
openjdk version "21.0.11" 2026-04-21
OpenJDK Runtime Environment (build 21.0.11+10-1-24.04.2-Ubuntu)
OpenJDK 64-Bit Server VM (build 21.0.11+10-1-24.04.2-Ubuntu, mixed mode, sharing)
Maven:
Apache Maven 3.8.7
Maven home: /usr/share/maven
Java version: 21.0.11, vendor: Ubuntu, runtime: /usr/lib/jvm/java-21-openjdk-amd64
Default locale: en, platform encoding: UTF-8
OS name: "linux", version: "6.17.0-1017-aws", arch: "amd64", family: "unix"
Docker:
Docker version 29.1.3, build 29.1.3-0ubuntu3~24.04.2
AWS CLI:
aws-cli/2.36.30 Python/3.14.6 Linux/6.17.0-1017-aws script-exe/x86_64.ubuntu.24
kubectl:
Client Version: v1.36.4
Kustomize Version: v5.8.1
Helm:
version.BuildInfo{Version:"v3.21.4", GitCommit:"813176c51bb5c181dbbd7901298ddcc104cd3417", GitTreeState:"clean", GoVersion:"go1.26.5"}
Ansible:
ansible [core 2.16.3]
  config file = None
  configured module search path = ['/root/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /root/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Mar 23 2026, 19:04:32) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.2
  libyaml = True
=====
IMPORTANT:
Log out and log back in after Docker group changes.
Then verify Jenkins can run Docker commands.
```

- After launching the Jenkins EC2 instance, I executed the setup-jenkins-ubuntu.sh script to automatically install and configure all the required DevOps tools. The script installed Java, Maven, Docker, AWS CLI, kubectl, Helm, and Ansible, and I verified their versions to ensure the Jenkins server was ready to build, test, and deploy the Digital Banking application

SS 6: Jenkins Service & Docker Verification

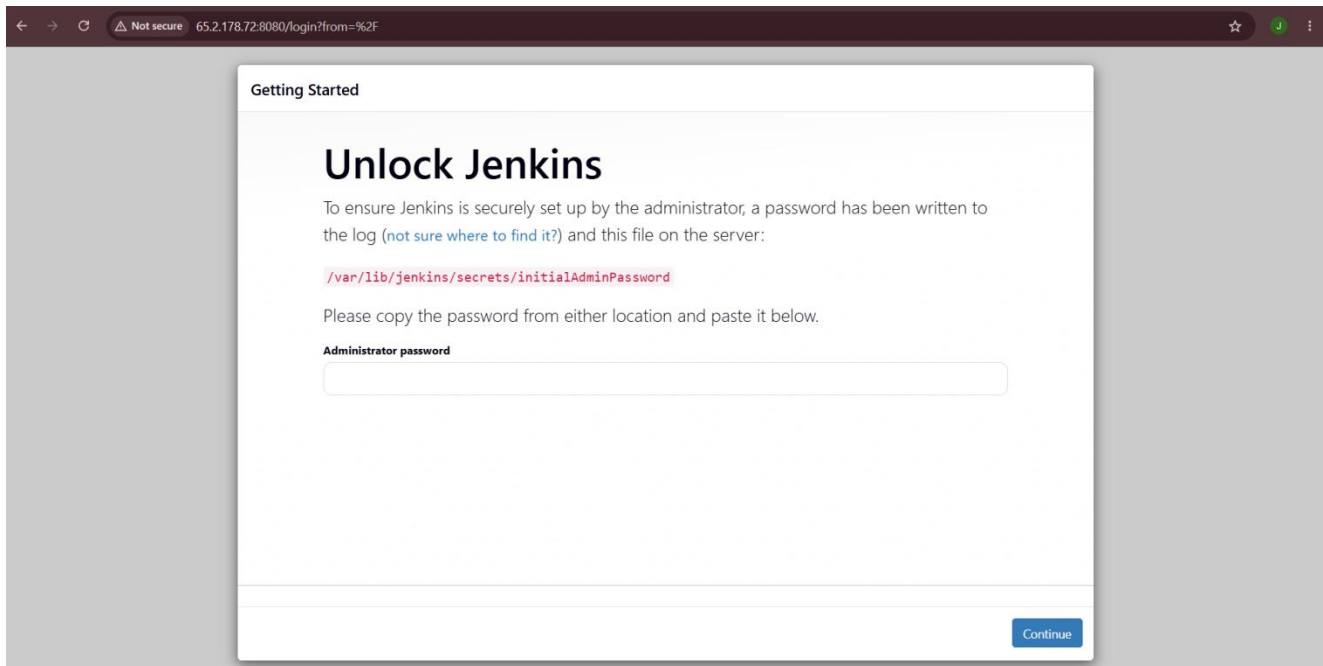
```
ubuntu@ip-10-0-15-121:~/digital-banking-cicd$ sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-08-24 22:27:43 UTC; 2min 2s ago
     Main PID: 14728 (java)
       Tasks: 43 (limit: 9269)
      Memory: 492.1M (peak: 495.2M)
         CPU: 12.469s
    CGroup: /system.slice/jenkins.service
            └─14728 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8080

Aug 24 22:27:38 ip-10-0-15-121 jenkins[14728]: [LF]> c0bdd5418a324db380cff4db4c236e46
Aug 24 22:27:38 ip-10-0-15-121 jenkins[14728]: [LF]>
Aug 24 22:27:38 ip-10-0-15-121 jenkins[14728]: [LF]> This may also be found at: /var/lib/jenkins/secrets/initialAdminPassword
Aug 24 22:27:38 ip-10-0-15-121 jenkins[14728]: [LF]>
Aug 24 22:27:38 ip-10-0-15-121 jenkins[14728]: [LF]> *****
Aug 24 22:27:38 ip-10-0-15-121 jenkins[14728]: [LF]> *****
Aug 24 22:27:38 ip-10-0-15-121 jenkins[14728]: [LF]> *****
Aug 24 22:27:43 ip-10-0-15-121 jenkins[14728]: 2026-08-24 22:27:43.146+0000 [id=38] INFO jenkins.InitReactorRunner$1#onAttained: Completed init
Aug 24 22:27:43 ip-10-0-15-121 jenkins[14728]: 2026-08-24 22:27:43.172+0000 [id=30] INFO hudson.lifecycle.Lifecycle#onReady: Jenkins is fully u
Aug 24 22:27:43 ip-10-0-15-121 systemd[1]: Started jenkins.service - Jenkins Continuous Integration Server.

ubuntu@ip-10-0-15-121:~/digital-banking-cicd$ sudo -u jenkins docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS   NAMES
ubuntu@ip-10-0-15-121:~/digital-banking-cicd$ sudo systemctl is-enabled jenkins
enabled
ubuntu@ip-10-0-15-121:~/digital-banking-cicd$ |
```

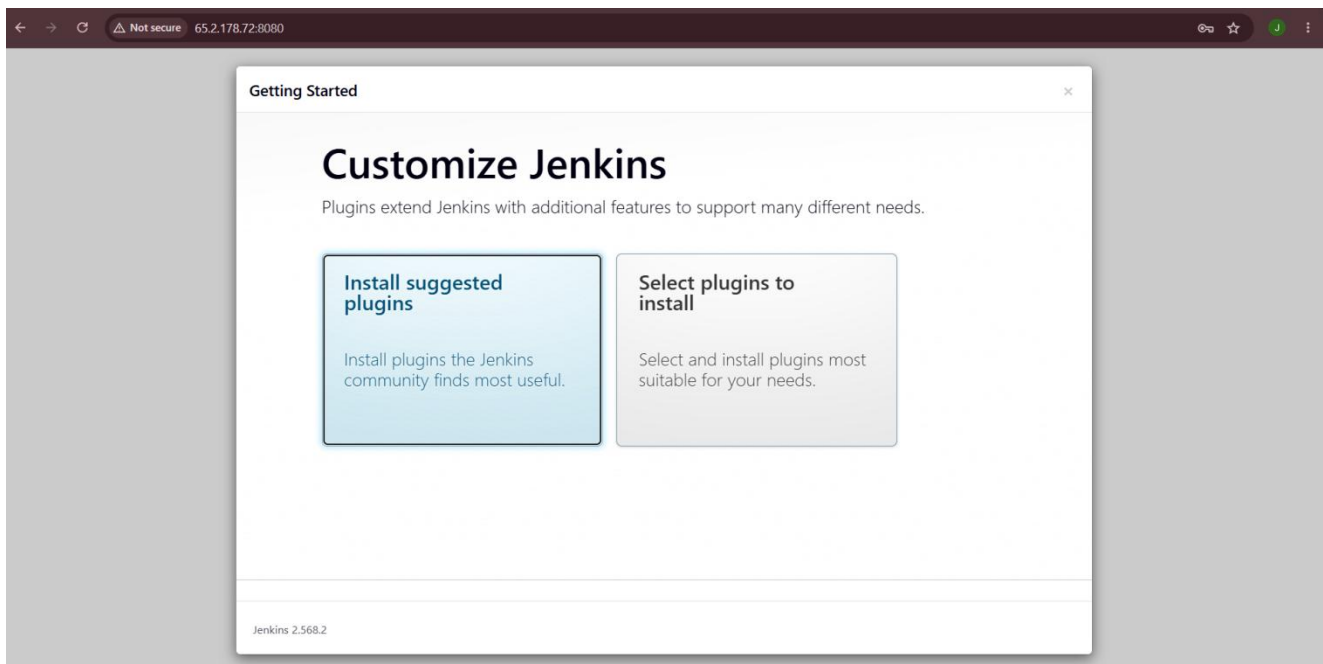
- After installing the required tools, I verified that the **Jenkins service** was running successfully on the EC2 server. I also verified that Jenkins could access **Docker** and enabled the Jenkins service to start automatically after a system restart. This confirmed that the Jenkins server was ready for CI/CD configuration.

SS 7: Jenkins Initial Setup and Unlock Page



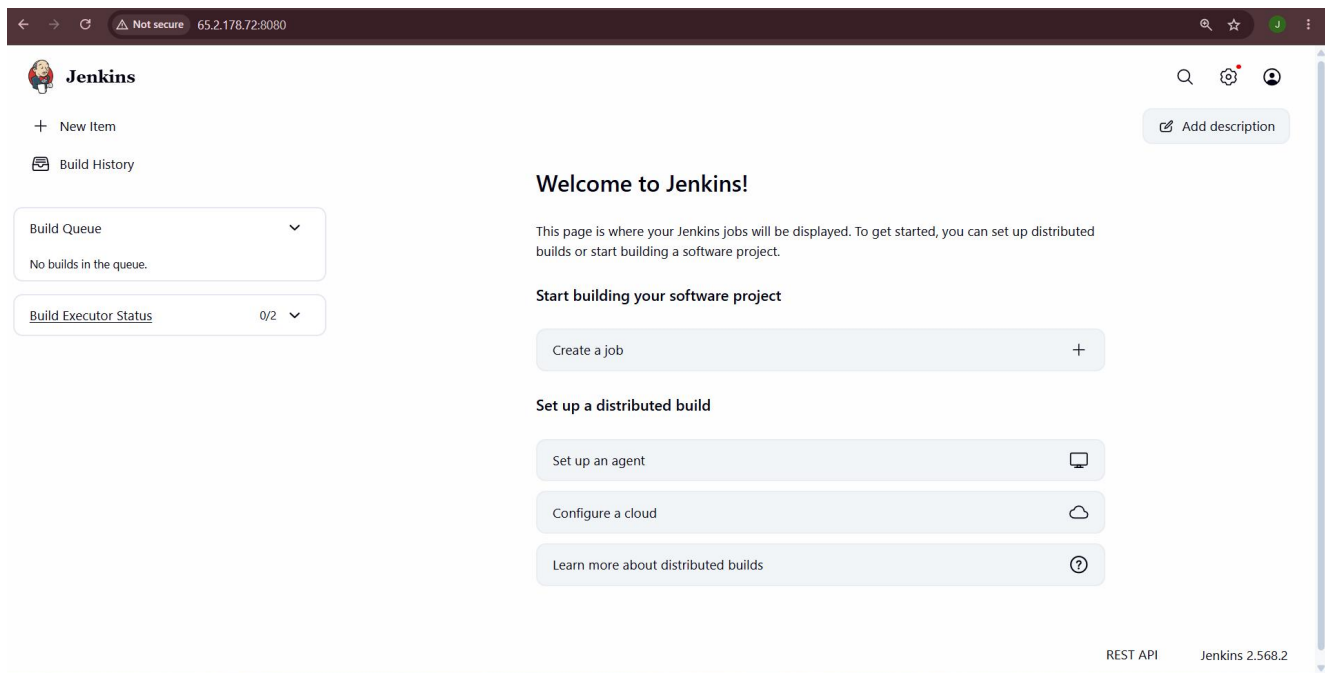
- After verifying that the Jenkins service was running, I accessed Jenkins through the browser using **port 8080**. The **Unlock Jenkins** page appeared, where I used the initial administrator password generated during the Jenkins installation to complete the initial setup.

SS 8: Jenkins Plugin Configuration



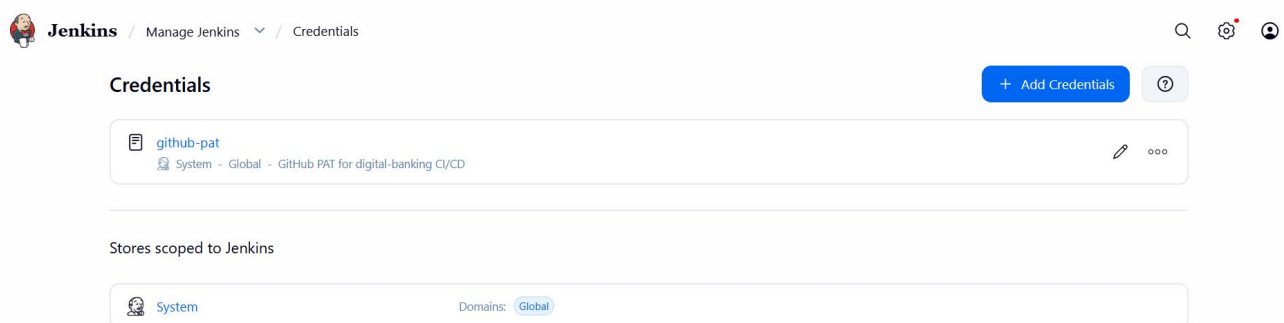
- After unlocking Jenkins, I configured the initial Jenkins environment by selecting the **Install suggested plugins** option. These plugins provide the basic functionality required to configure and run the Jenkins CI/CD pipeline.

SS 9: Jenkins Dashboard



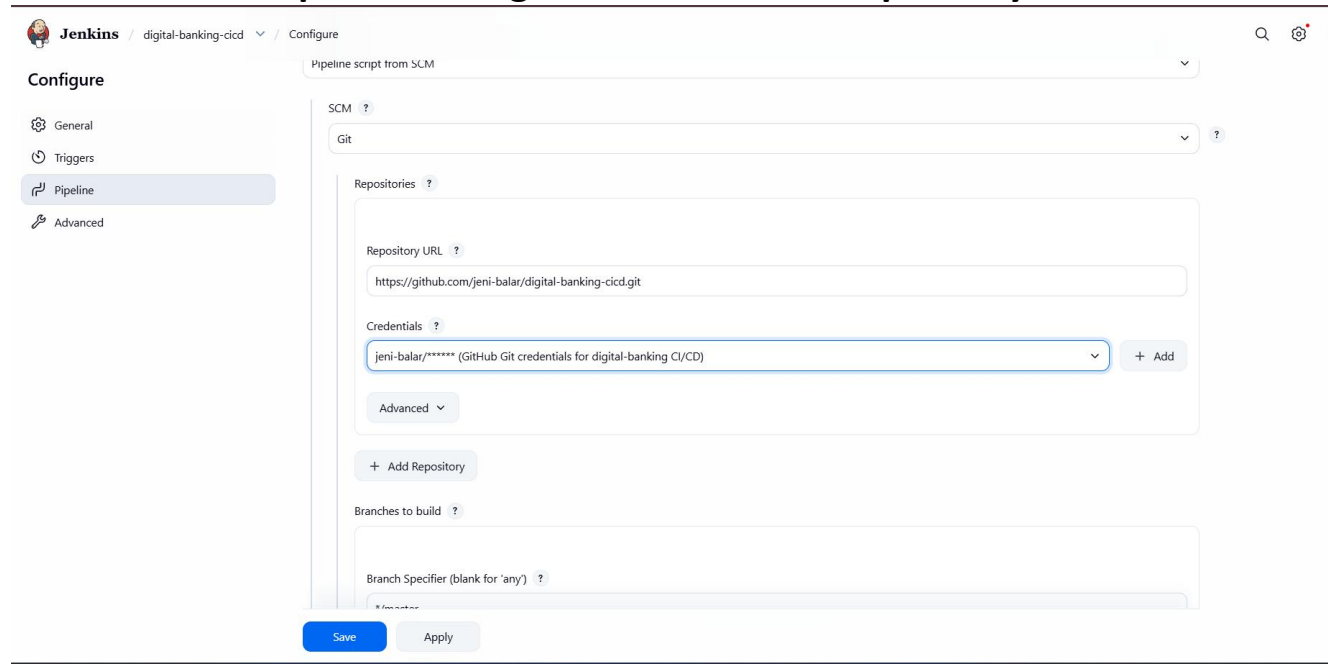
- After completing the initial Jenkins configuration and installing the required plugins, I accessed the **Jenkins dashboard** successfully. The dashboard confirmed that Jenkins was ready to create jobs and configure the **Digital Banking CI/CD pipeline**

SS 10: GitHub Personal Access Token Stored as Jenkins Credential



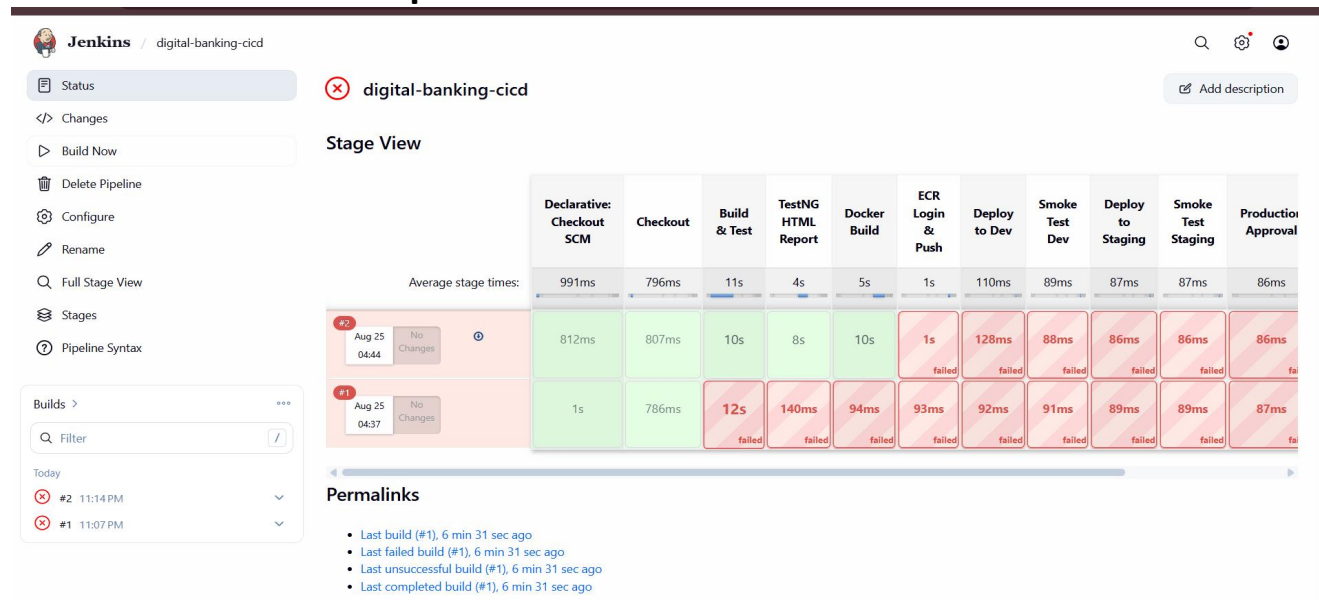
- Then I configured a **GitHub Personal Access Token (PAT)** in the Jenkins Credentials Manager and saved it with the ID github-pat. This credential allows Jenkins to securely authenticate with the GitHub repository and access the Digital Banking project source code without storing the GitHub password directly in the pipeline.

SS 11: Jenkins Pipeline Configuration – GitHub Repository



- Then I configured the digital-banking-cicd Jenkins pipeline to use **Git** as the SCM and connected it to the Digital Banking GitHub repository. I selected the configured GitHub credentials and set the **master branch**. The pipeline uses the **Jenkinsfile** stored in the **GitHub repository**, allowing Jenkins to read and execute the pipeline stages defined in the project.

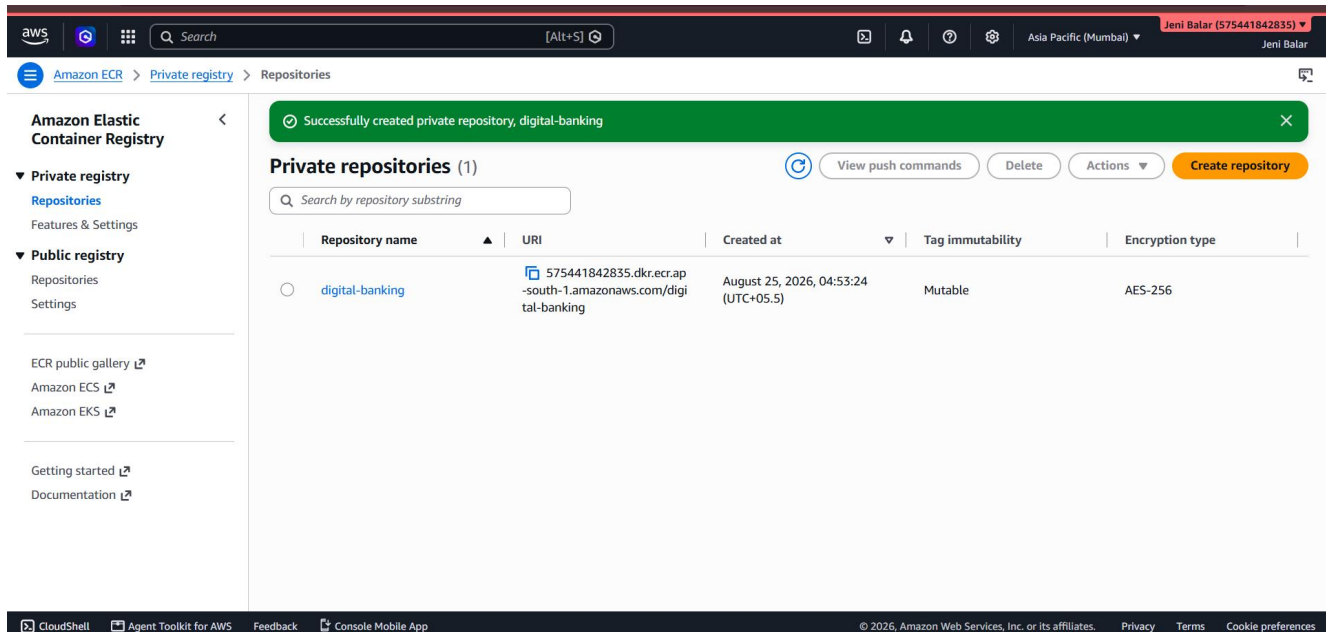
SS 12: Jenkins Initial Pipeline Build & Docker Verification



- Then I executed the digital-banking-cicd pipeline to verify the initial CI/CD configuration. The pipeline successfully completed the stages up to **Docker Build**, confirming that source checkout, Maven build and testing, TestNG reporting, and Docker image creation were working correctly. The remaining stages failed because the **ECR**, **deployment**, and **environment configurations** had not yet been completed.

Docker & Amazon ECR

SS 13: Private Amazon ECR Repository Created



- I created a private **Amazon ECR repository** named **digital-banking** to store the Docker images generated by the Jenkins pipeline. The repository was created in the **Mumbai (ap-south-1) region** with **AES-256 encryption**. Jenkins later uses this repository to push the application Docker image for deployment to Amazon EKS.

SS 14: Successful Docker Login to Amazon ECR

```
ubuntu@ip-10-0-15-121:~$ aws ecr get-login-password --region ap-south-1 | \
docker login --username AWS --password-stdin \
575441842835.dkr.ecr.ap-south-1.amazonaws.com

WARNING! Your credentials are stored unencrypted in '/home/ubuntu/.docker/config.json'.
Configure a credential helper to remove this warning. See
https://docs.docker.com/go/credential-store/

Login Succeeded
ubuntu@ip-10-0-15-121:~$ |
```

- Then I configured Docker authentication with the private **Amazon ECR registry** using the AWS CLI. The `aws ecr get-login-password` command generated a temporary authentication token, which was passed to Docker for registry login. The **“Login Succeeded”** message confirmed that the Jenkins EC2 server could authenticate successfully with Amazon ECR.

SS 15: Jenkins Docker Access & Image Verification

```
ubuntu@ip-10-0-15-121:~$ groups
ubuntu adm cdrom sudo dip lxd docker
ubuntu@ip-10-0-15-121:~$ docker images

IMAGE                                ID                                DISK USAGE    CONTENT SIZE  EXTRA
575441842835.dkr.ecr.ap-south-1.amazonaws.com/digital-banking:2 a594201f32eb                    474MB         133MB
digital-banking:2                  a594201f32eb                    474MB         133MB
eclipse-tomcat:17-jre              13cc28a6cc72                   430MB         114MB
ubuntu@ip-10-0-15-121:~$ sudo usermod -aG docker jenkins
ubuntu@ip-10-0-15-121:~$ sudo systemctl restart jenkins
ubuntu@ip-10-0-15-121:~$ sudo -u jenkins docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS        NAMES
```

- To allow Jenkins to run Docker commands, I added the jenkins **user to the Docker group** and restarted the Jenkins service. I then verified Docker access using `sudo -u jenkins docker ps` and checked the available Docker images on the server. This ensured that Jenkins had the required Docker access for building and working with the Digital Banking application image.

SS 16: Docker Image Successfully Pushed to Amazon ECR

```
ubuntu@ip-10-0-15-121:~$ docker images

IMAGE                                ID                                DISK USAGE    CONTENT SIZE  EXTRA
575441842835.dkr.ecr.ap-south-1.amazonaws.com/digital-banking:2 a594201f32eb                    474MB         133MB
digital-banking:2                  a594201f32eb                    474MB         133MB
eclipse-tomcat:17-jre              13cc28a6cc72                   430MB         114MB
ubuntu@ip-10-0-15-121:~$ docker push 575441842835.dkr.ecr.ap-south-1.amazonaws.com/digital-banking:2
The push refers to repository [575441842835.dkr.ecr.ap-south-1.amazonaws.com/digital-banking]
409c221e1962: Pushed
d12223f877f0: Pushed
ae7a97d182d7: Pushed
1a085c661b8f: Pushed
06e9d71331fb: Pushed
f3db1cd94078: Pushed
074b156e16dd: Pushed
c9d311702c92: Pushed
2: digest: sha256:a594201f32eb902ae640351245b9e67d03c923b8452d61b2514fbd5c3df3030c size: 2051
ubuntu@ip-10-0-15-121:~$
```

- Then I pushed the digital-banking:2 **Docker image** to the private **Amazon ECR repository** using the repository URI. The output showed that all image layers were successfully pushed and the image was stored in ECR with the **2 tag**. This confirmed that the Docker image was ready to be used for deployment to Amazon EKS.

SS 17: Amazon ECR Docker Image Verification

The screenshot shows the Amazon ECR console interface. On the left, there's a navigation menu with 'Amazon Elastic Container Registry' and sections for 'Private registry' and 'Public registry'. The main area displays the 'digital-banking' repository. The 'Images' tab is active, showing a table with one image entry. The table has columns for 'Image tags', 'Type', 'Created at', 'Image size (MB)', 'Image digest', and 'Last pulled at'. The image '2' is listed with a creation time of 'August 25, 2026, 04:59:38 (UTC+05.5)', a size of '132.58' MB, and a digest of 'sha256:a594201f32eb902ae640351245b9e67d03c923b8452d61b2514fbd5c3df3030c'. Above the table, there are buttons for 'Delete', 'Copy URI', 'Details', 'Scan', and 'View push commands'. The top of the console shows the AWS logo, a search bar, and the user 'Jeni Balar' with their account ID '575441842835'.

- After pushing the Docker image from the Jenkins server, I verified the digital-banking repository in Amazon ECR. The repository showed the uploaded image with the **2 tag**, confirming that the Docker image was successfully stored in ECR and was ready for deployment to **Amazon EKS**.

Amazon EKS & Application Verification

SS 18: Amazon EKS Cluster Active – Digital Banking

The screenshot displays the Amazon Elastic Kubernetes Service (EKS) console for the 'digital-banking-eks' cluster. The cluster is in an 'Active' status, indicated by a green checkmark. Key details include:

- Cluster info:** Status is Active, Kubernetes version is 1.36, Support period is Standard support until August 2, 2027, and Provider is EKS.
- Cluster health:** 0 issues.
- Upgrade insights:** 4 insights.
- Node health issues:** 0 issues.
- Capability issues:** 0 issues.

The 'Details' section provides further configuration information:

- API server endpoint:** <https://FE57A0898DB533CDF9F62BF6486B97DB.r7.ap-south-1.eks.amazonaws.com>
- OpenID Connect provider URL:** <https://oidc.eks.ap-south-1.amazonaws.com/id/FE57A0898DB533CDF9F62BF6486B97DB>
- Certificate authority:** LS0tLS1CRUdJTiBDRVJUSUZQOFUR50tLS0tCk1JSURkNDQWZjZ0F3SUJBZ0lSQUt4emhK1FvNTVJVElobkFrb3dNUF3RFFZSktvWk
- Cluster IAM role ARN:** [arn:aws:iam::575441842835:role/AmazonEKSClusterRole](#)
- Cluster ARN:** [arn:aws:eks:ap-south-1:575441842835:cluster/digital-banking-eks](#)
- Platform version:** info

- I created the digital-banking-eks **Amazon EKS cluster** to provide the Kubernetes environment for deploying the Digital Banking application. The cluster was successfully created with **Kubernetes version 1.36** and showed an **Active** status with no cluster or node health issues. The cluster IAM role and **OIDC provider** were also configured for the EKS environment.

SS 19: EKS Managed Node Group & Worker Nodes

The screenshot displays the Amazon Elastic Kubernetes Service (EKS) console for the 'digital-banking-nodes' managed node group. The node group is in an 'Active' status, indicated by a green checkmark. Key details include:

- Node group configuration:** Kubernetes version is 1.36, AMI type is Amazon Linux 2023 (x86_64) Standard, AMI release version is 1.36.2-20260818, Instance types are c7i-flex.large, and Disk size is 20 GiB.

The 'Nodes' section shows two worker nodes in a 'Ready' state:

Node name	Instance type	Managed by	Created	Status	CPU usage	Memory	
ip-10-0-129-125.ap-south-1.compute.internal	c7i-flex.large	Node group	digital-banking-nodes	3 minutes ago	Ready	12%	18%
ip-10-0-155-76.ap-south-1.compute.internal	c7i-flex.large	Node group	digital-banking-nodes	3 minutes ago	Ready	32%	32%

- Then I created the digital-banking-nodes **managed node group** for the EKS cluster and configured the worker-node capacity, including the **desired number of nodes**. The node group uses **Amazon Linux 2023** with c7i-flex.large instances, and both worker nodes were in the **Ready state**, providing the compute resources required to run the Digital Banking application Pods.

SS 20: EKS Worker Nodes & Kubernetes Pods Verification

```
ubuntu@ip-10-0-15-121:~$ kubectl get nodes
NAME                                STATUS    ROLES    AGE    VERSION
ip-10-0-129-125.ap-south-1.compute.internal Ready    <none>   22m    v1.36.2-eks-b3f9404
ip-10-0-155-76.ap-south-1.compute.internal Ready    <none>   22m    v1.36.2-eks-b3f9404
ubuntu@ip-10-0-15-121:~$ kubectl get pods -A
NAMESPACE      NAME                                READY    STATUS    RESTARTS   AGE
kube-system    aws-node-2g8q5                     2/2      Running   0           24m
kube-system    aws-node-bndvc                     2/2      Running   0           23m
kube-system    coredns-574bf556c4-k61s8          1/1      Running   0           82m
kube-system    coredns-574bf556c4-sdm8d          1/1      Running   0           82m
kube-system    eks-pod-identity-agent-4xbrf       1/1      Running   0           23m
kube-system    eks-pod-identity-agent-7f12s       1/1      Running   0           24m
kube-system    kube-proxy-sbb68                   1/1      Running   0           24m
kube-system    kube-proxy-xhj75                   1/1      Running   0           23m
kube-system    metrics-server-54675c6bc8-4q7m9   1/1      Running   0           82m
kube-system    metrics-server-54675c6bc8-4v8gq   1/1      Running   0           82m
ubuntu@ip-10-0-15-121:~$
```

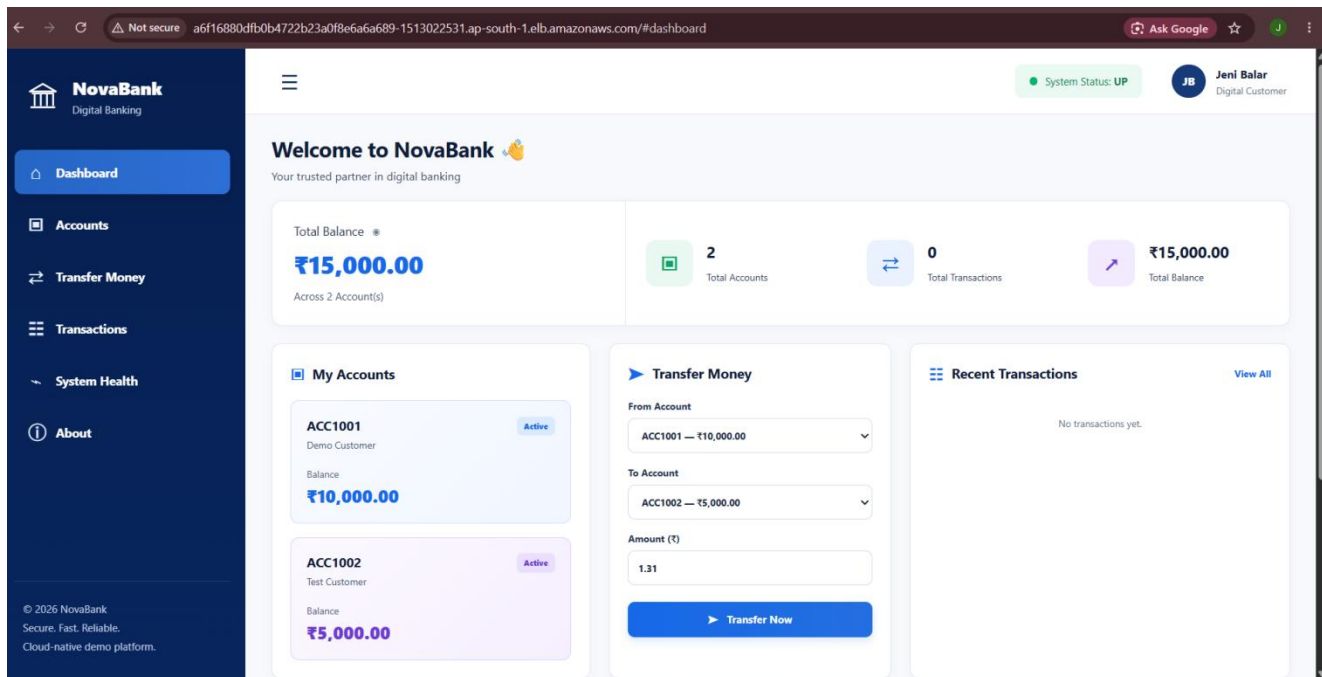
- After creating the EKS node group, I verified the cluster using kubectl. The output showed **two EKS worker nodes in the Ready state**, confirming that they were successfully connected to the cluster. I also checked all Pods across the cluster, and the required **Kubernetes system Pods were running successfully** with no restarts.

SS 21: Digital Banking Application Deployment on EKS

```
ubuntu@ip-10-0-15-121: ~/digital-banking-cicd
ubuntu@ip-10-0-15-121:~/digital-banking-cicd$ kubectl apply -f k8s/deployment.yaml
deployment.apps/digital-banking created
service/digital-banking created
ubuntu@ip-10-0-15-121:~/digital-banking-cicd$ kubectl get pods
NAME                                READY    STATUS    RESTARTS   AGE
digital-banking-6d977df668-qxqmd    0/1      Running   0           12s
digital-banking-6d977df668-rhrhr    0/1      Running   0           12s
ubuntu@ip-10-0-15-121:~/digital-banking-cicd$ kubectl get pods
NAME                                READY    STATUS    RESTARTS   AGE
digital-banking-6d977df668-qxqmd    1/1      Running   0           84s
digital-banking-6d977df668-rhrhr    1/1      Running   0           84s
ubuntu@ip-10-0-15-121:~/digital-banking-cicd$ |
```

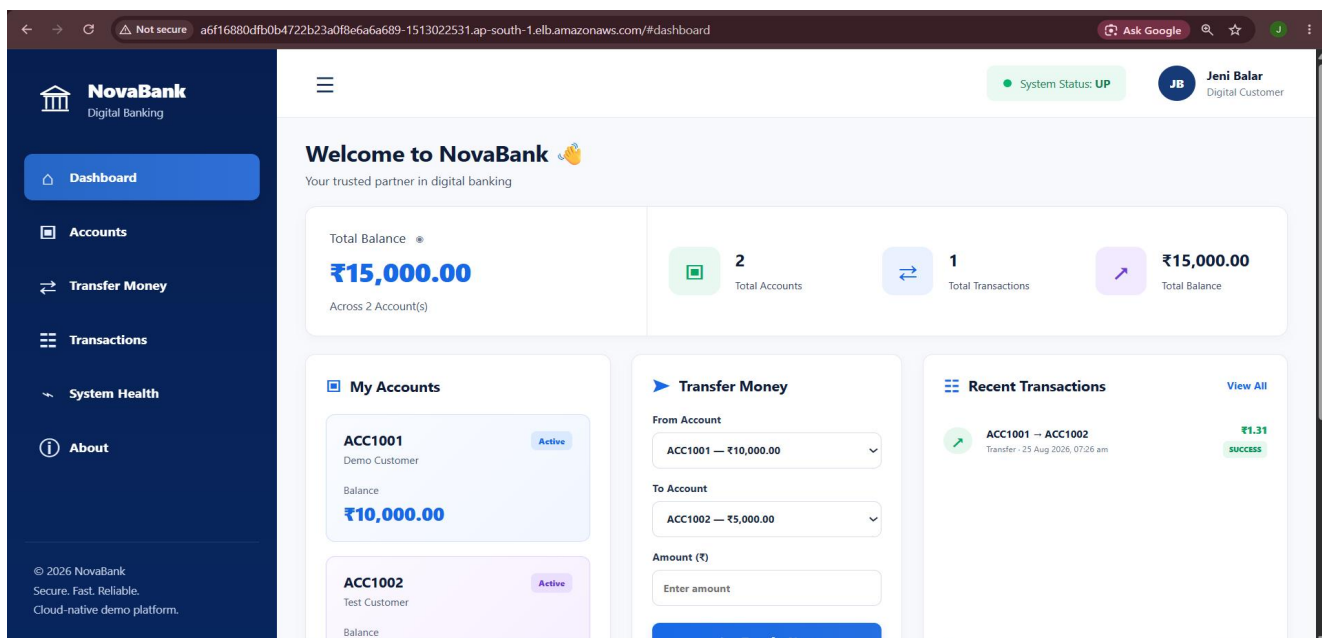
- Then I deployed the **Digital Banking application** to the EKS cluster using the Kubernetes manifest file k8s/deployment.yaml. The Deployment and Service were created successfully, and I verified the application Pods using kubectl get pods. Both **Digital Banking Pods reached the 1/1 Running state**, confirming that the application was successfully deployed on EKS.

SS 22: Digital Banking Application – LoadBalancer Verification



- After the application Pods were running successfully, I accessed the **Digital Banking application** using the **Kubernetes LoadBalancer endpoint**. The application dashboard loaded successfully, displaying features such as **Accounts**, **Transfer Money**, **Transactions**, and **System Health**. I also entered an amount in the **Transfer Money** section to verify the application's transaction functionality

SS 23: Digital Banking – Successful Transaction Verification



- To verify the application's functionality, I performed a **money transfer from ACC1001 to ACC1002** using the Transfer Money feature. The transaction was completed successfully, and the dashboard updated the **Total Transactions** count to **1** and displayed the successful transaction under **Recent Transactions**. This confirmed that the deployed application was not only accessible but also performing its banking functionality correctly.

SS 24: Digital Banking – Application Health Check

```
ubuntu@ip-10-0-15-121:~/digital-banking-cicd$ curl http://a6f16880dfb0b4722b23a0f8e6a6a689-1513022531.ap-south-1.elb.amazonaws.com/actuator/health
{"status":"UP","groups":["liveness","readiness"]}ubuntu@ip-10-0-15-121:~/digital-banking-cicd$
ubuntu@ip-10-0-15-121:~/digital-banking-cicd$
```

- Then verified the health of the deployed application using the **Spring Boot Actuator health endpoint**. The response returned "status":"UP", confirming that the Digital Banking application was running successfully and its health checks were passing.

Kubernetes Deployment & Monitoring Setup

SS 25: Kubernetes Deployment and LoadBalancer Verification

```
ubuntu@ip-10-0-15-121:~/digital-banking-cicd$ kubectl get deployment
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
digital-banking 2/2     2            2          32m
ubuntu@ip-10-0-15-121:~/digital-banking-cicd$ kubectl get svc
NAME          TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
digital-banking  LoadBalancer  172.20.197.225   a6f16880dfb0b4722b23a0f8e6a6a689-1513022531.ap-south-1.elb.amazonaws.com  80:32415/TCP    32m
kubernetes    ClusterIP      172.20.0.1      <none>           443/TCP          129m
ubuntu@ip-10-0-15-121:~/digital-banking-cicd$
```

- I verified the Kubernetes deployment using `kubectl get deployment` and `kubectl get svc`. The **digital-banking deployment** shows **2/2 pods ready and available**, while the **digital-banking service** is configured as a **LoadBalancer** and provides the external AWS LoadBalancer endpoint on **port 80**.

SS 26: Kubernetes Rollout and Resource Status Verification

```
ubuntu@ip-10-0-15-121:~/digital-banking-cicd$ kubectl rollout status deployment/digital-banking
deployment "digital-banking" successfully rolled out
ubuntu@ip-10-0-15-121:~/digital-banking-cicd$ kubectl get all
NAME                                READY   STATUS    RESTARTS   AGE
pod/digital-banking-6d977df668-qxqmd 1/1     Running   0          33m
pod/digital-banking-6d977df668-rhrhr 1/1     Running   0          33m
NAME                                TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
service/digital-banking             LoadBalancer  172.20.197.225   a6f16880dfb0b4722b23a0f8e6a6a689-1513022531.ap-south-1.elb.amazonaws.com  80:32415/TCP    33m
service/kubernetes                   ClusterIP      172.20.0.1      <none>           443/TCP          130m
NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/digital-banking      2/2     2            2          33m
NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/digital-banking-6d977df668 2         2         2       33m
ubuntu@ip-10-0-15-121:~/digital-banking-cicd$
```

- I verified the deployment rollout using `kubectl rollout status`, which confirmed that the **digital-banking deployment was successfully rolled out**. The `kubectl get all` command also showed **2/2 pods running**, the **LoadBalancer service active**, and the deployment with **2 available replicas**. This confirmed that the application was successfully deployed and running on the EKS cluster.

SS 27: Digital Banking – Prometheus Metrics Verification

```
ubuntu@ip-10-0-15-121:~/digital-banking-cicd$ kubectl exec deployment/digital-banking -- curl -s http://localhost:8080/actuator/prometheus | head -20
# HELP application_ready_time_seconds Time taken for the application to be ready to service requests
# TYPE application_ready_time_seconds gauge
application_ready_time_seconds{application="digital-banking",main_application_class="com.jeni.banking.DigitalBankingApplication"} 12.675
# HELP application_started_time_seconds Time taken to start the application
# TYPE application_started_time_seconds gauge
application_started_time_seconds{application="digital-banking",main_application_class="com.jeni.banking.DigitalBankingApplication"} 12.474
# HELP disk_free_bytes Usable space for path
# TYPE disk_free_bytes gauge
disk_free_bytes{application="digital-banking",path="/app/."} 1.7193951232E10
# HELP disk_total_bytes Total space for path
# TYPE disk_total_bytes gauge
disk_total_bytes{application="digital-banking",path="/app/."} 2.1395124224E10
# HELP executor_active_threads The approximate number of threads that are actively executing tasks
# TYPE executor_active_threads gauge
executor_active_threads{application="digital-banking",name="applicationTaskExecutor"} 0.0
# HELP executor_completed_tasks_total The approximate total number of tasks that have completed execution
# TYPE executor_completed_tasks_total counter
executor_completed_tasks_total{application="digital-banking",name="applicationTaskExecutor"} 0.0
# HELP executor_pool_core_threads The core number of threads for the pool
# TYPE executor_pool_core_threads gauge
ubuntu@ip-10-0-15-121:~/digital-banking-cicd$ AC
ubuntu@ip-10-0-15-121:~/digital-banking-cicd$
```

- Then verified the application's **Prometheus metrics endpoint** using the Kubernetes deployment. The endpoint returned metrics such as **application readiness time**, **startup time**, **disk usage**, **active threads**, and **completed tasks**. These metrics can be collected by **Prometheus** and later used in **Grafana dashboards** for monitoring the Digital Banking application.

SS 28: Kubernetes Monitoring Stack – Prometheus and Grafana Pods

```
ubuntu@ip-10-0-15-121:~/digital-banking-cicd$ kubectl get pods -n monitoring
NAME                                READY   STATUS    RESTARTS   AGE
alertmanager-kube-prometheus-stack-alertmanager-0    2/2     Running   0           36s
kube-prometheus-stack-grafana-74ccfdb78-j4dpp        3/3     Running   0           44s
kube-prometheus-stack-kube-state-metrics-5f6764465d-jgjww  1/1     Running   0           44s
kube-prometheus-stack-operator-5fc657d788-4t6gr      1/1     Running   0           44s
kube-prometheus-stack-prometheus-node-exporter-c6xvf  1/1     Running   0           44s
kube-prometheus-stack-prometheus-node-exporter-1sb94  1/1     Running   0           44s
prometheus-kube-prometheus-stack-prometheus-0       2/2     Running   0           37s
ubuntu@ip-10-0-15-121:~/digital-banking-cicd$ |
```

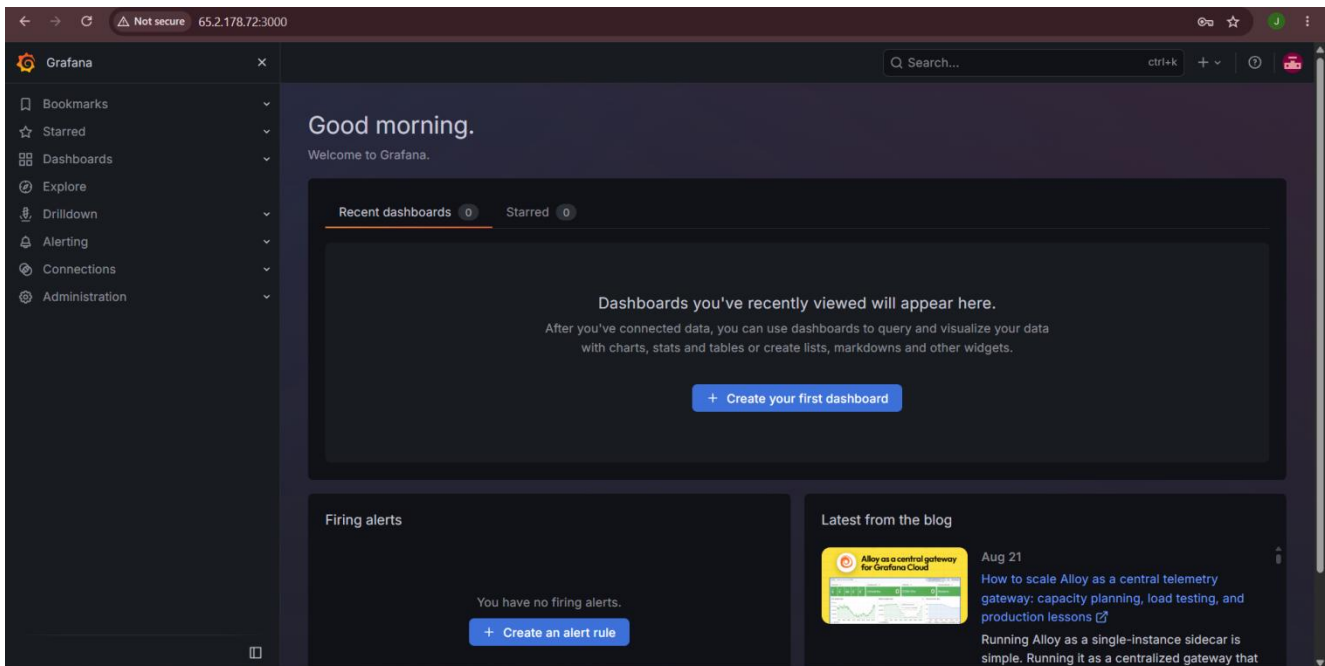
- I then verified the monitoring components deployed in the monitoring **namespace**. The output shows **Prometheus, Grafana, Alertmanager, Kube State Metrics, Prometheus Operator, and Node Exporter** pods in the **Running** state. This confirmed that the monitoring stack was successfully deployed and ready to collect and visualize metrics from the Kubernetes environment.

SS 29: Prometheus – Digital Banking Active Target Verification

```
ubuntu@ip-10-0-15-121:~/digital-banking-cicd$ curl -s http://localhost:9090/api/v1/targets | python3 -c 'import sys,json; d=json.load(sys.stdin); [(print(x["labels"].get("job"), x["health"], x["scrapeUrl"])) for x in d["data"]["activeTargets"] if "digital-banking" in str(x["labels"])]'
digital-banking up http://10.0.152.235:8080/actuator/prometheus
digital-banking up http://10.0.137.161:8080/actuator/prometheus
ubuntu@ip-10-0-15-121:~/digital-banking-cicd$ |
```

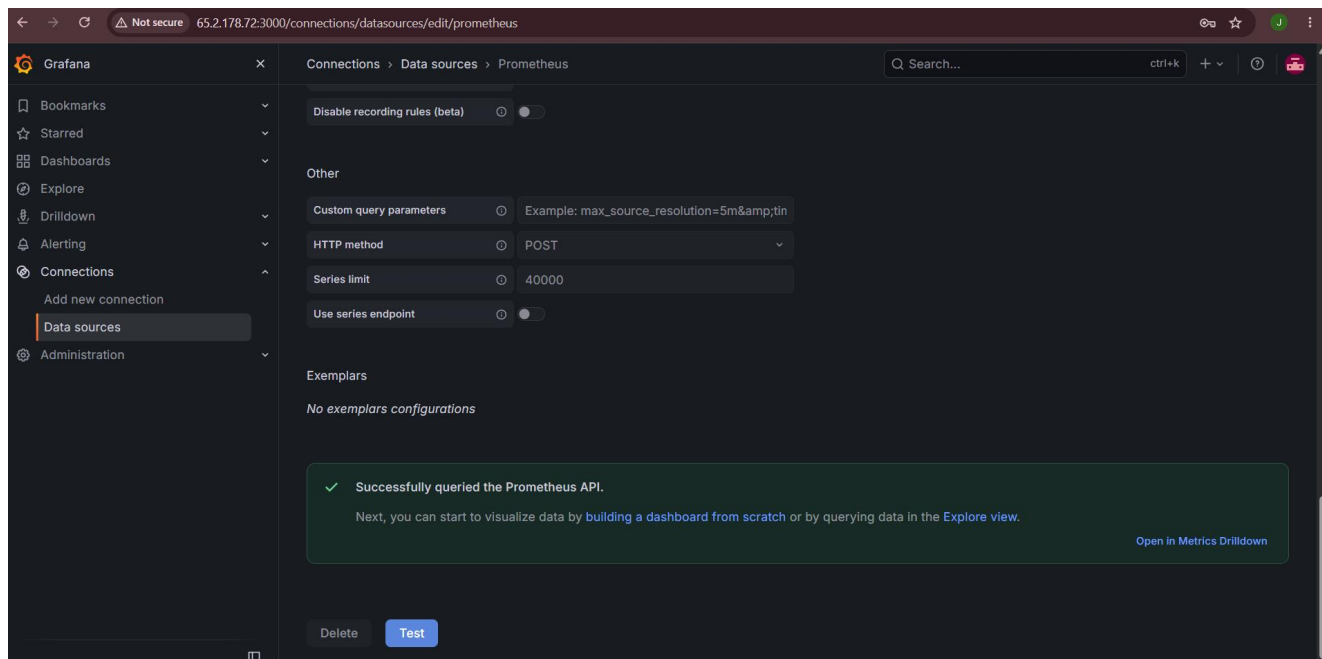
- I verified that the **Prometheus active target** for the Digital Banking application. The output shows the digital-banking target with the **UP** health status and the `/actuator/prometheus` endpoint, confirming that Prometheus is successfully collecting metrics from the deployed application.

SS 30: Grafana Monitoring Dashboard Interface



- Then accessed **Grafana** through its web interface to set up the monitoring dashboard. Grafana was successfully accessible, and the dashboard interface was ready to visualize the metrics collected by **Prometheus** from the Digital Banking application.

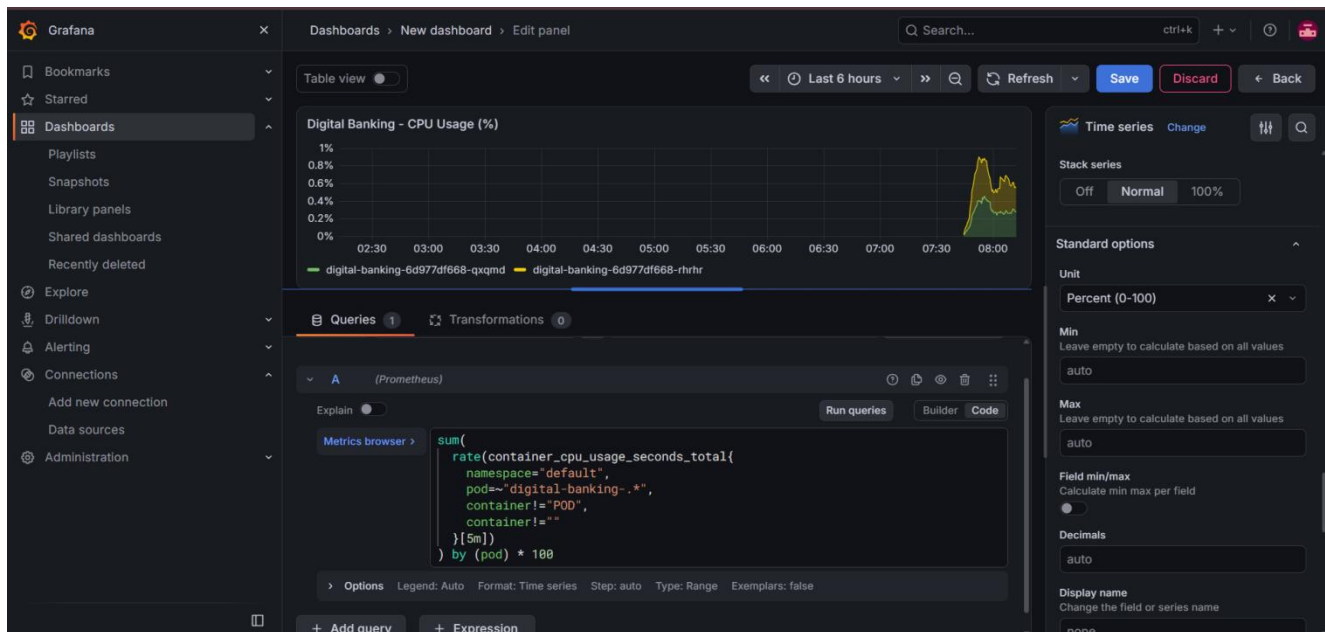
SS 31: Grafana – Prometheus Data Source Connection



- I configured **Prometheus** as the data source in **Grafana** and tested the connection. Grafana successfully queried the **Prometheus API**, confirming that the connection was working correctly and that the collected application metrics were available for creating monitoring dashboards.

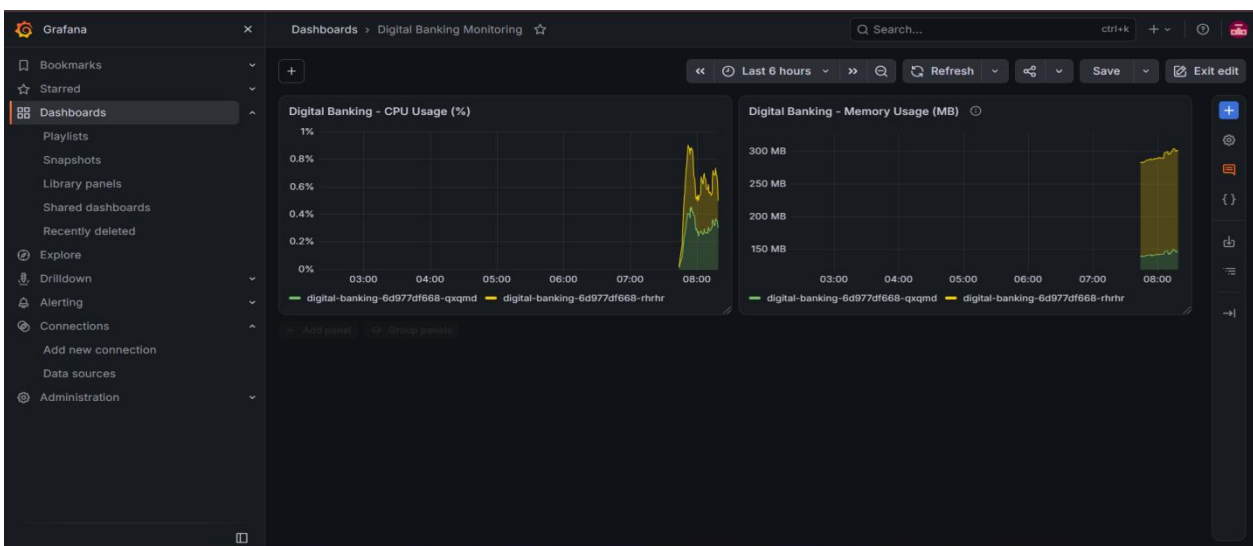
Grafana Monitoring Dashboards

SS 32: Grafana – Digital Banking CPU Usage Panel



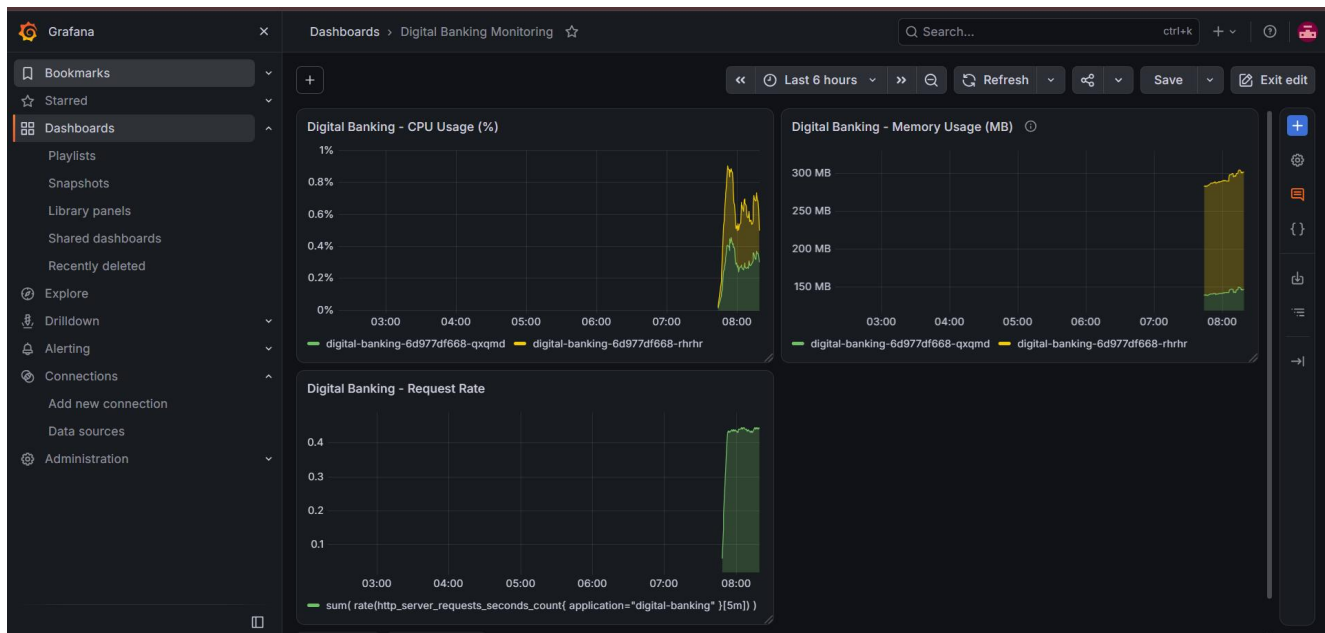
- I created a **CPU Usage** panel in Grafana using **Prometheus** as the data source. The panel uses a PromQL query to calculate the CPU usage of the **Digital Banking application Pods** and displays the results as a time-series graph. This allows CPU usage to be monitored over time for the deployed application.

SS 33: Grafana – Digital Banking Memory Usage Panel



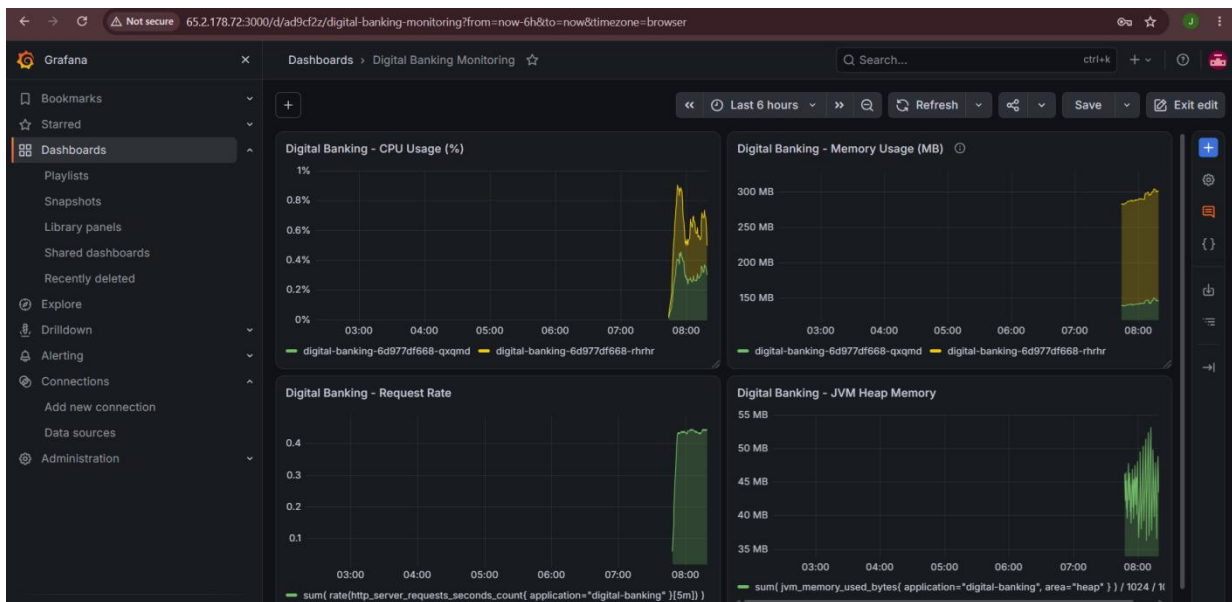
- Then I created a **Memory Usage** panel in Grafana using **Prometheus** as the data source. The panel uses a PromQL query to calculate the memory usage of the **Digital Banking application Pods** and displays the results as a time-series graph. This allows the memory consumption of the deployed application to be monitored over time.

SS 34: Grafana – Digital Banking Request Rate Panel



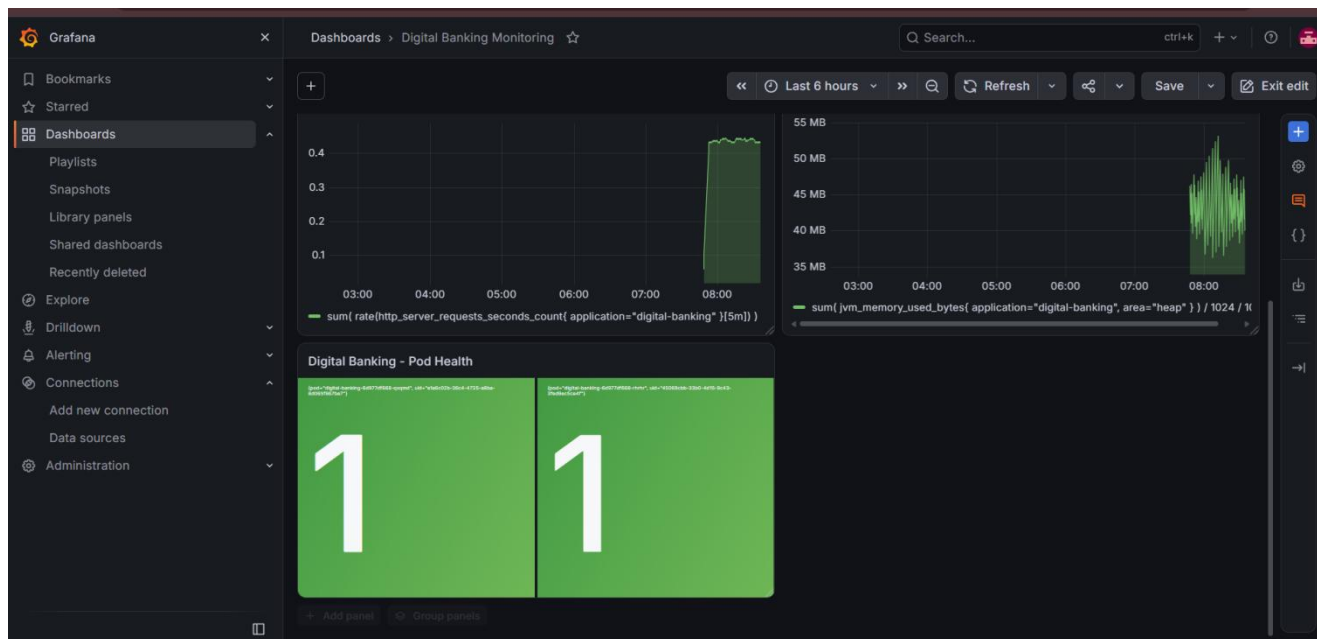
- I Next, I added a **Request Rate** panel to the **Digital Banking Monitoring** dashboard. Using **Prometheus** metrics, the panel tracks the rate of HTTP requests handled by the application and displays the request activity over time. This helps in observing the application's traffic and request handling performance.

SS 35: Grafana – Digital Banking JVM Heap Memory Panel



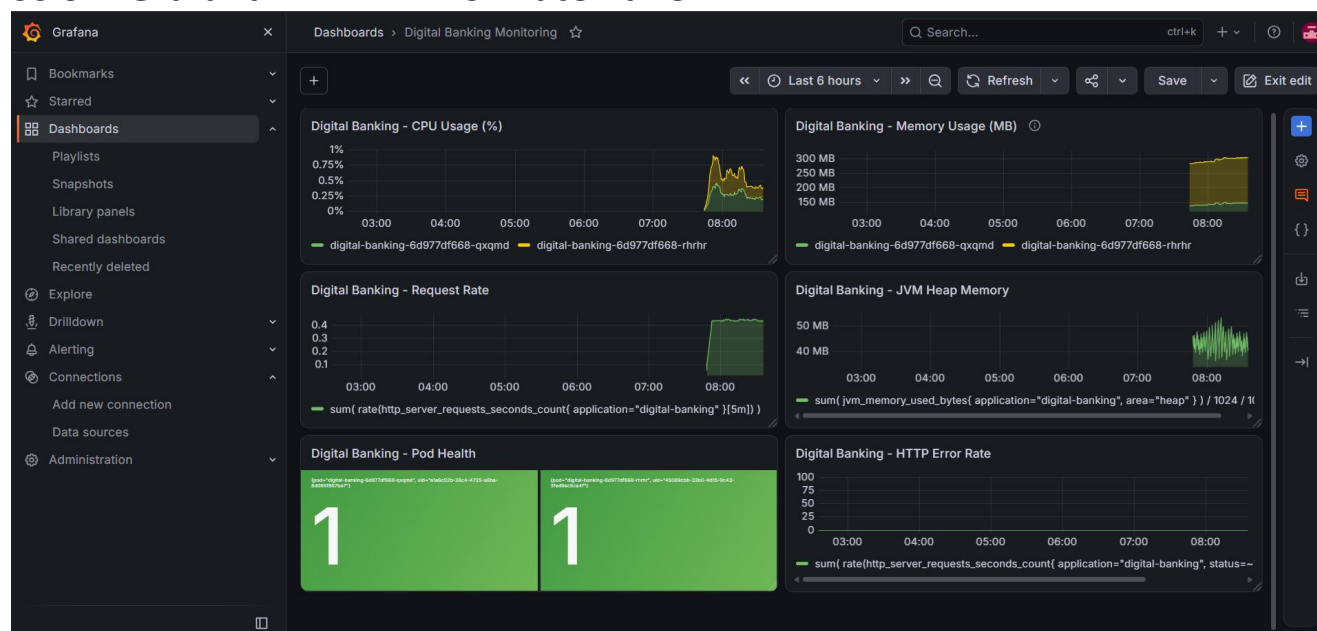
- I then added a **JVM Heap Memory** panel to the **Digital Banking Monitoring** dashboard. The panel uses **Prometheus** metrics to visualize the application's heap memory consumption over time. This provides visibility into JVM memory usage and helps identify unusual memory growth or potential memory-related issues.

SS 36: Grafana – Kubernetes Pod Health Panel



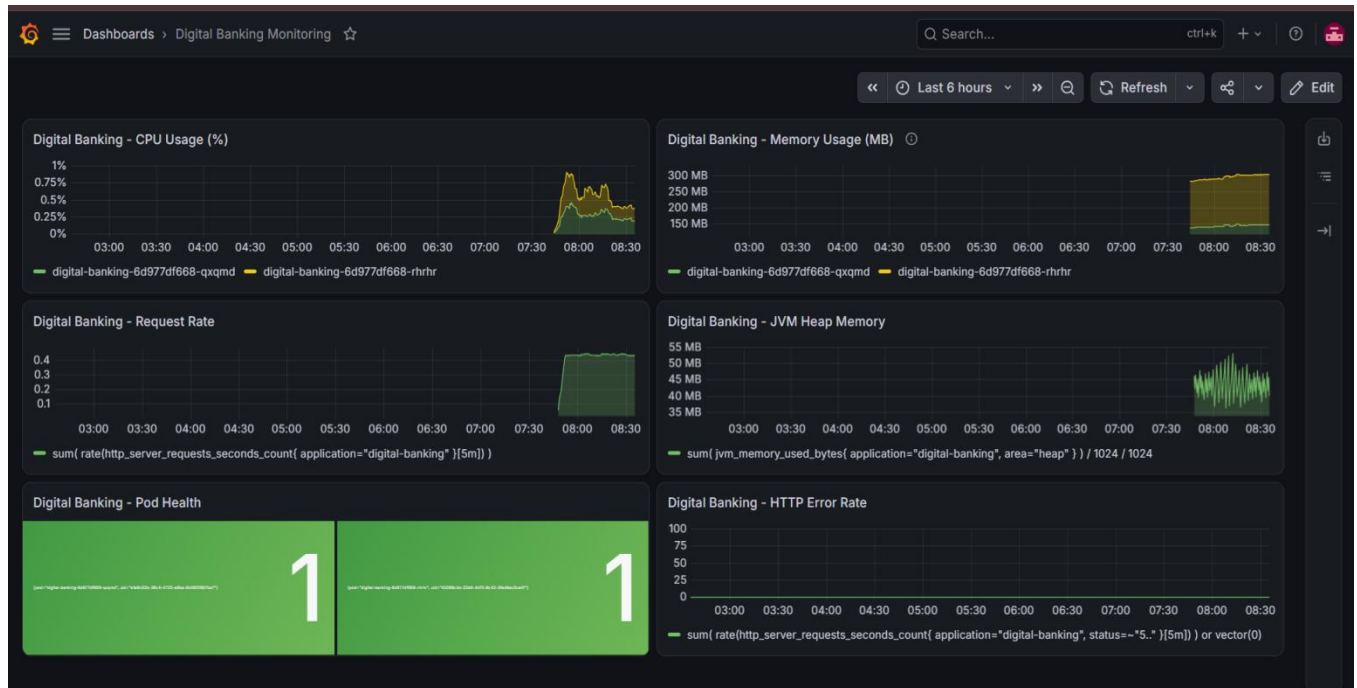
- I added a **Kubernetes Pod Health** panel to the **Digital Banking Monitoring** dashboard. This panel displays the health status of the application's Kubernetes pods, with each pod shown as **1 (healthy/available)**. The green status indicates that both deployed pods are currently operating normally.

SS 37: Grafana – HTTP Error Rate Panel



- I added **HTTP Error Rate** panel to the **Digital Banking Monitoring** dashboard. This panel tracks the rate of HTTP error responses generated by the application over time. The graph currently remains close to **0**, indicating that no significant HTTP errors are being observed during the monitored period.

SS 38: Final Grafana – Digital Banking Monitoring Dashboard



- Finally, I completed the **Digital Banking Monitoring** dashboard in Grafana by bringing all six monitoring panels together in a single view: **CPU Usage**, **Memory Usage**, **Request Rate**, **JVM Heap Memory**, **Kubernetes Pod Health**, and **HTTP Error Rate**. This provides a centralized overview of the application's performance, resource utilization, pod health, and error status for easier monitoring and analysis.

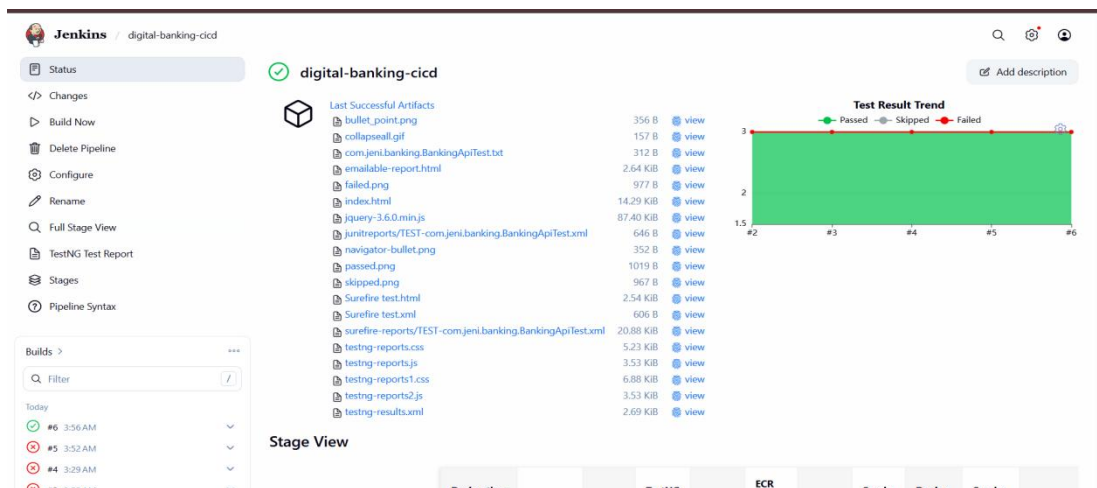
Final CI/CD Verification

SS 39: Final Jenkins – Complete CI/CD Pipeline Execution



- Finally, I successfully executed the complete **Jenkins CI/CD pipeline** for the Digital Banking application. The pipeline completed all stages successfully, including **SCM Checkout**, **Build & Test**, **TestNG HTML Report**, **Docker Build**, **ECR Login & Push**, **Dev Deployment**, **Dev Smoke Test**, **Staging Deployment**, **Staging Smoke Test**, **Production Approval**, **Production Deployment**, and **Production Smoke Test**. The successful **Build #6** confirms that the application was built, tested, containerized, pushed to Amazon ECR, and deployed successfully through the complete deployment flow.

SS 40: Jenkins – Test Results & Build Artifacts



- The Jenkins pipeline also generated and archived the **test reports and build artifacts** from the successful execution. The dashboard displays the **Test Result Trend**, showing consistent test results across builds #2 to #6, along with the available **TestNG reports**, **Surefire reports**, **XML results**, and **other generated artifacts**. This confirms that the automated testing results were successfully collected and made available through Jenkins.

5. Kubernetes and Deployment Design

The Digital Banking application is packaged as a Docker image and deployed to Amazon EKS. The Kubernetes deployment defines the application workload, while the Kubernetes Service of type LoadBalancer provides the single external application endpoint. The deployment process is automated through Ansible so that the Jenkins pipeline can update the EKS environment consistently.

Deployment Sequence

- AWS CLI updates the kubeconfig for the digital-banking-eks cluster.
- The deployment namespace is created if it does not already exist.
- k8s/deployment.yaml is applied to the selected namespace.
- The deployment image is updated to the ECR image URI generated from the Jenkins build tag.
- Kubernetes rollout status is checked with a deployment timeout.

Environment Selection

The Ansible playbook maps environments to Kubernetes namespaces: **dev** → **banking-dev**, **staging** → **banking-staging**, and **prod** → **banking-prod**. Jenkins supplies the environment and image tag as deployment variables.

Autoscaling

The EKS worker-node capacity is backed by AWS Auto Scaling through the node-group infrastructure. This allows the worker capacity to be managed as part of the EKS environment while Kubernetes schedules the application pods onto available nodes.

6. Monitoring and Observability

Prometheus and Grafana are used to monitor the Digital Banking environment. The application exposes metrics that can be scraped by Prometheus, while Grafana uses Prometheus as its data source and presents the metrics through dashboards.

- Prometheus metrics endpoint verified.
- Prometheus target scraping verified.
- Grafana accessed successfully.
- Prometheus configured as the Grafana data source.
- CPU usage dashboard created/verified.
- Memory usage dashboard created/verified.
- Request-rate panel created/verified.
- JVM heap memory panel created/verified.
- Kubernetes pod-health monitoring verified.
- HTTP error-rate panel created/verified.
- Final Digital Banking monitoring dashboard completed.

7. Key Verification Commands

- `git ls-tree -r HEAD --name-only | grep '^k8s/'`
- `pwd && ls -la k8s/deployment.yaml`
- `aws sts get-caller-identity`
- `aws eks update-kubeconfig --region ap-south-1 --name digital-banking-eks`
- `kubectl get nodes`
- `kubectl get pods -n banking-dev`
- `kubectl get pods -n banking-staging`
- `kubectl get pods -n banking-prod`
- `kubectl get svc -n banking-prod`
- `kubectl get deployment -n banking-prod`
- `kubectl rollout status deployment/digital-banking -n banking-prod`
- `kubectl describe pods -n banking-prod`
- `sudo systemctl status jenkins`
- `docker images`
- `aws ecr describe-images --repository-name digital-banking --region ap-south-1`
- `kubectl get all -n banking-prod`
- `curl http://<LOAD_BALANCER_DNS>/actuator/health`
- `kubectl exec deployment/digital-banking -n banking-prod -- curl -s http://localhost:8080/actuator/prometheus`
- `curl -s http://localhost:9090/api/v1/targets`
- `kubectl get pods -n monitoring`

8. Troubleshooting

1. Kubernetes Manifest Path Issue

During deployment, Ansible failed because `kubect` could not find `k8s/deployment.yaml`, even though the file existed in the Jenkins workspace. I verified the workspace and manifest location and identified that the issue was caused by the relative file path. The Ansible playbook was updated to use `playbook_dir` to reference the Kubernetes manifest correctly. After this change, the deployment completed successfully and the application was running on Amazon EKS.

2. ECR Authentication and Image Push Issue

Before deploying the application to EKS, Docker needed to authenticate with the private **Amazon ECR** repository. I configured ECR authentication using the AWS CLI and verified the Docker login successfully. The Digital Banking Docker image was then pushed to ECR and verified in the repository, making it available for the EKS deployment.

9. End-to-End Verification

The complete Digital Banking CI/CD implementation was verified through the following checks:

- GitHub repository is successfully configured as the source for the Jenkins pipeline.
- Jenkins successfully performs source checkout and application build and test stages.
- TestNG test results are generated and available in Jenkins.
- Docker image is successfully built and pushed to the private **Amazon ECR** repository.
- Amazon EKS cluster and the managed worker node group are active and ready.
- Digital Banking application Pods are in the **Running/Ready** state.
- Kubernetes LoadBalancer Service successfully provides an external AWS endpoint.
- Digital Banking application is accessible through the browser.
- A **money transfer transaction** was successfully performed and verified through the deployed application.
- Spring Boot **Actuator health check** returns the application status as **UP**.
- Prometheus successfully detects the Digital Banking application target and collects its metrics.
- Grafana successfully connects to Prometheus and displays the monitoring dashboards.
- The final Jenkins Stage View confirms that the complete CI/CD pipeline, including **Production Approval and Production Smoke Test**, completed successfully.

10. Conclusion

The **Digital Banking CI/CD project** successfully demonstrates an end-to-end DevOps workflow for deploying a Spring Boot application on AWS. The project integrates **GitHub, Jenkins, Maven, TestNG, Docker, Amazon ECR, Ansible, Kubernetes, Amazon EKS, Prometheus, and Grafana** to automate application build, testing, containerization, deployment, validation, and monitoring.

The application was successfully deployed across **Dev, Staging, and Production environments**, with Kubernetes providing container orchestration and a LoadBalancer Service providing external access. The implementation also established monitoring through Prometheus and Grafana, enabling visibility into application performance, resource usage, request activity, JVM memory, Pod health, and HTTP errors.

Overall, the project demonstrates practical knowledge of **CI/CD automation, AWS cloud infrastructure, containerization, Kubernetes deployment, deployment troubleshooting, environment promotion, and application monitoring**, providing a complete foundation for managing and delivering applications in a cloud-based DevOps environment.